

♪ VGA Project

The Bremen Town Musicians

- 카메라와 FPGA가 함께 꿈을 이루는 연주 이야기 -

연주곡 · 뒸다 뒸다 비행기

대한상공회의소 서울기술교육센터

이준형, 곽은찬, 안정현, 윤지원, 최여지, 최은수

2026.07.21



INDEX

서곡 · Prelude

Introduction

프로젝트 배경 · 개발 목표 · 개발 환경



제 1 악장

Block Diagram

전체 시스템 구성 · 다중 FPGA 보드 연동 구조와 데이터 흐름



제 2 ~ 4 악장

Key Technologies

영상 기반 음 검출 · SPI 영상 전송 · PC 연주 애플리케이션



제 5 악장

UVM Verification

주요 기능 검증 환경 · 테스트 시나리오 · 검증 결과



제 6 악장

Troubleshooting

개발 과정의 주요 문제 · 원인 분석 · 해결 방법



피날레 · Finale

Demonstration & Reflections

뒹다뒹다 비행기 연주 시연 · 개발 결과 · 회고 및 Q&A





Introduction



서곡

Introduction

6대의 OV7670 카메라 영상을 FPGA에서 실시간으로 통합하고,
표식의 위치를 음계 신호로 변환하는 분산형 비전 연주 시스템

Project Objective

- Master 1대와 Slave 5대에 영상 처리 기능을 분산
- 5채널 병렬 SPI를 통해 5개의 영상 프레임을 동시에 전송
- 2x3 VGA 모자이크 영상과 12비트 음계 데이터를 실시간으로 출력

Development Environment



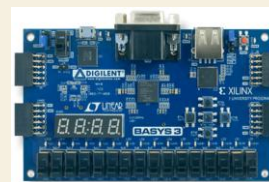
SystemVerilog

영상, 통신 RTL



Vivado 2020.2

합성, 구현, 검증



Basys3

FPGA



OV7670

QVGA 영상 입력

1 + 5

MASTER + SLAVE

6 CAM

OV7670 INPUT

5 CH

PARALLEL SPI

640 x 480

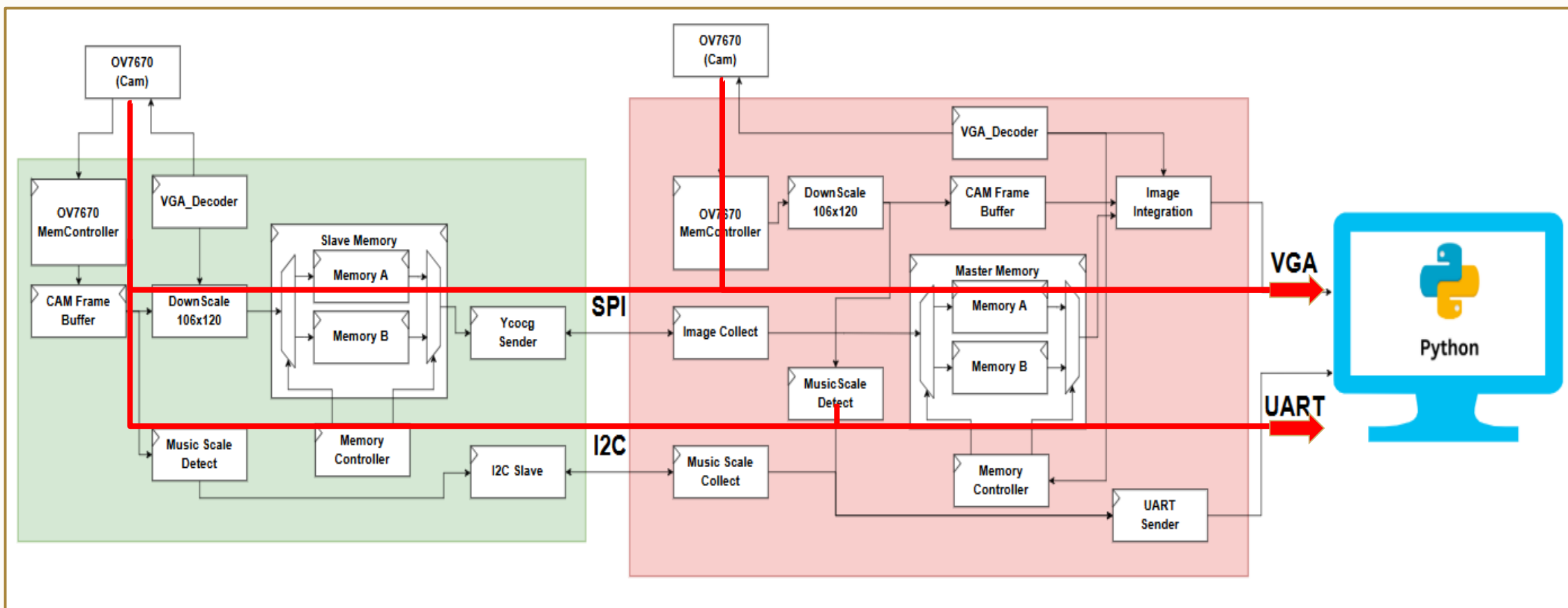
VGA MOSAIC



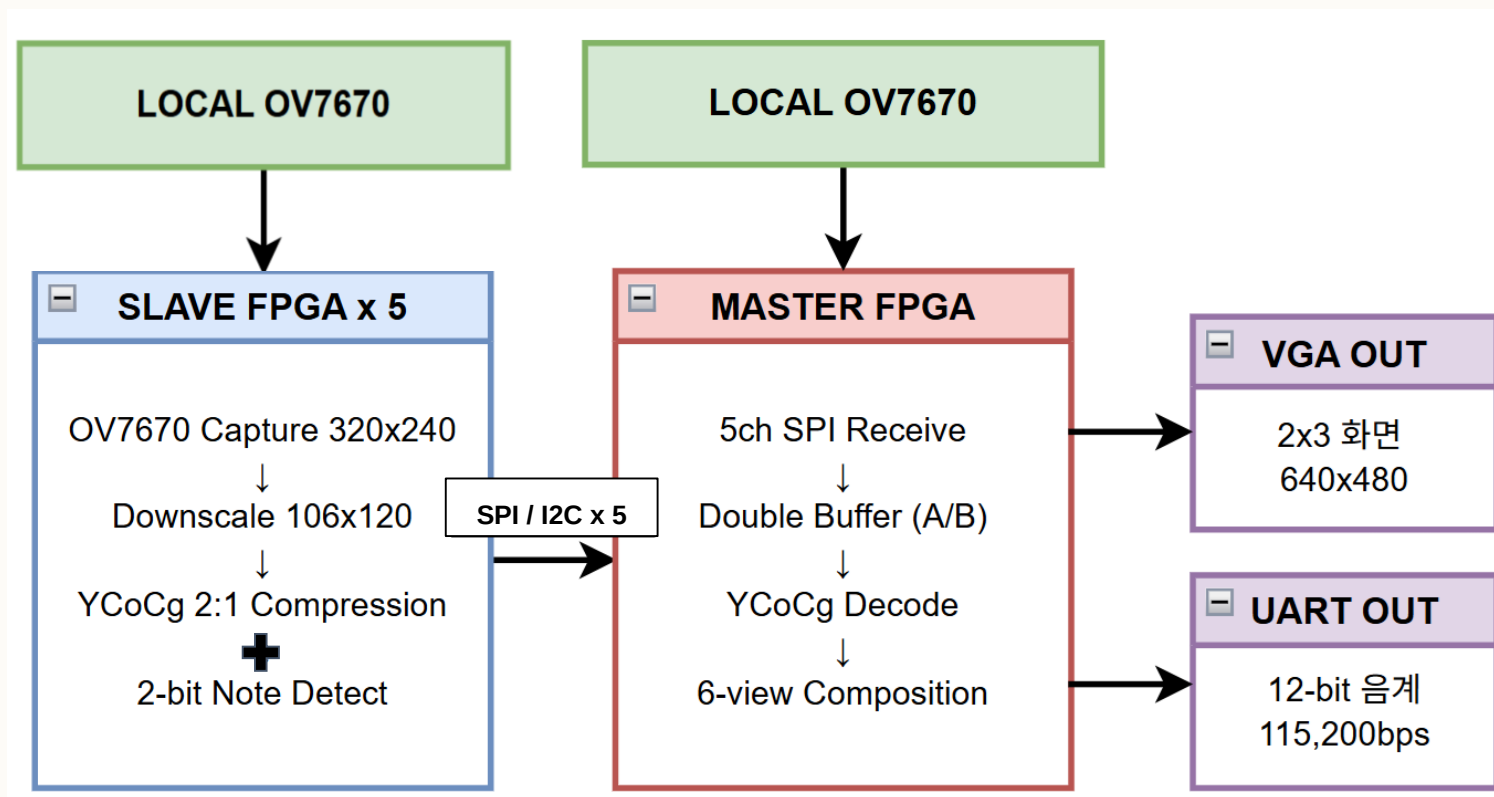
Block Diagram

♪ 제 1 악장

Block Diagram



Block Diagram



CAPTURE

각 보드가 OV7670 영상을 수집

DOWNSCALE

영상을 106x120 해상도로 축소

COMPRESS

YCoCg 2:1 압축으로 전송 데이터
50% 절감

COMPOSE

VGA 영상과 음계 데이터를 동시 출력

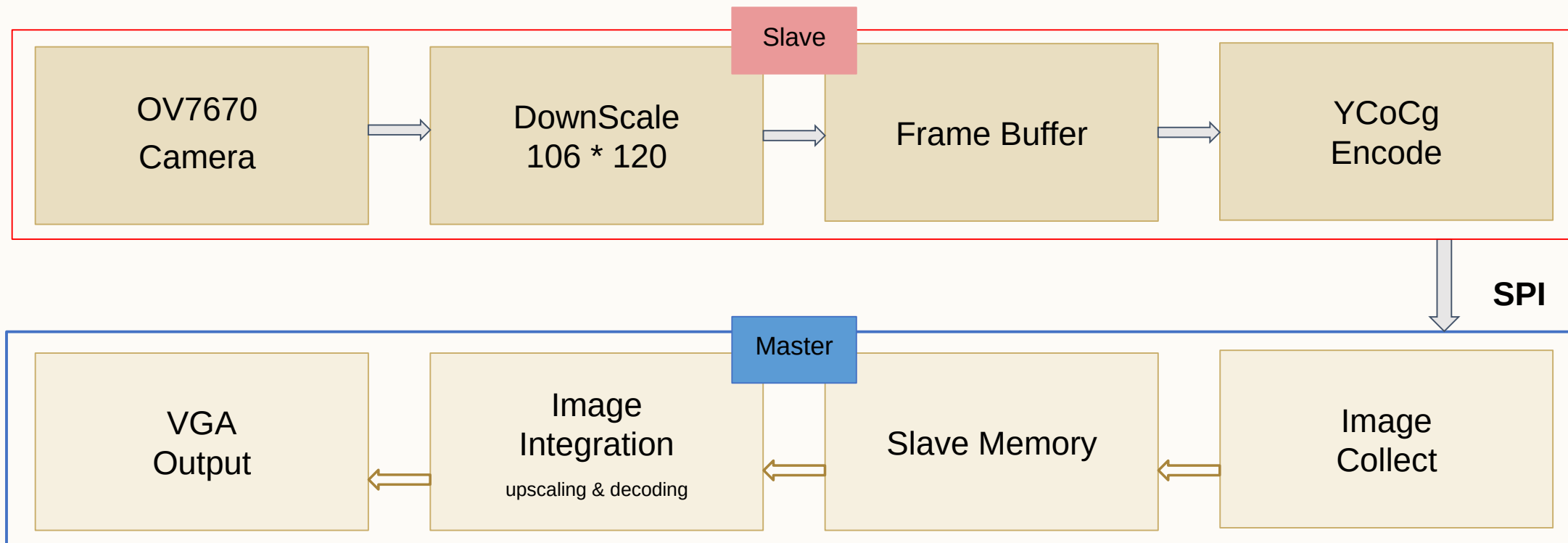
TRANSFER

5대의 Slave 영상을 병렬로 수신



Image Integration Flow

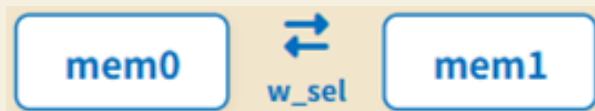
Image Integration Flow



1. OV7670에서 들어오는 데이터는 끊기지 않음
2. Basys3 보드의 메모리가 한정적

Image Flow - double buffering structure

핑퐁 제어 로직



1. 안전한 스왑

데이터 준비됨 (frame_done) & 송신 가능 (!tx_busy)일때만 버퍼 전환

2. 데이터 처리 보호

Processing 상태에서는 덮어쓰기 방지 (픽셀 단위 깨짐 방지)

3. 상호 동기화

데이터 준비됨 ↔ 처리 완료 핸드셰이크로 안정적인 제어

핑퐁 메모리 구조

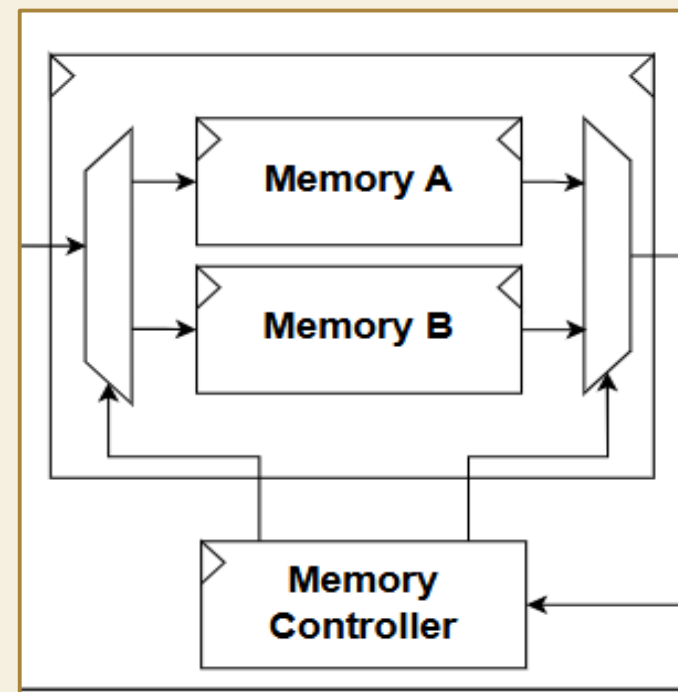


Image Flow - YCoCg Sender

픽셀 변환



$$Y = (R + 2G + B + 2) >> 2$$
$$Co_{hp} = R - B$$
$$Cg_{hp} = 2G - R - B$$

4픽셀 합산 후 양자화



$$sum_co = \sum Co_{hp} (4px)$$
$$Co = sum_co / 8$$
$$sum_cg = \sum Cg_{hp} (4px)$$
$$Cg = sum_cg / 16$$

24bit 패킷 구성



Y3[3:0]	Y2[3:0]	Y1[3:0]	Y0[3:0]	Co[3:0]	Cg[3:0]
< 패킷 구성 요약(총 24bit) >				< 부호 저장 트릭 >	
Co : 4bit				부호 있는 Co / Cg 데이터를 4bit	
Cg : 4bit				필드에 저장하기 위해	
Y0 ~ Y3 : 각 4bit (4*4=16bit)				offset (+8) 적용	



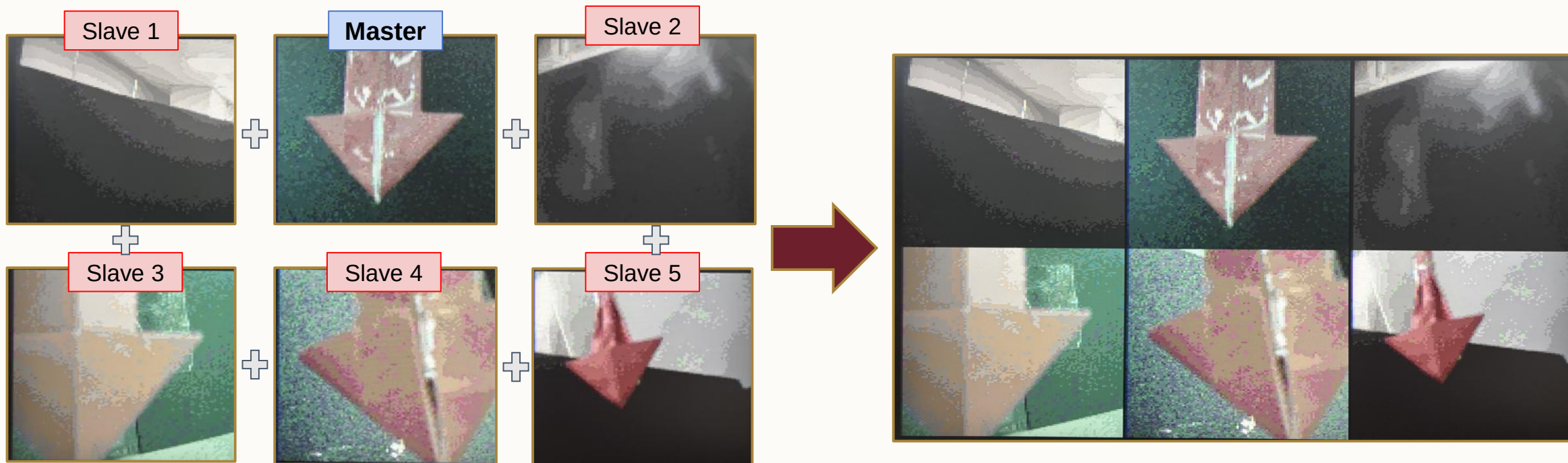
4 Original Pixels (RGB)



24bit Packet

대역폭 50% 감소

Image Flow - integration mosaic & decoding



< 데이터 복원 (Decoding) >

Slave의 24bit YCoCg
→ Master에서 48bit RGB로 복원

< 메모리 매핑 (Memory Mapping) >

6개 영상을 VGA Frame Buffer의 정확한 주소에
기록해 모자이크 구성

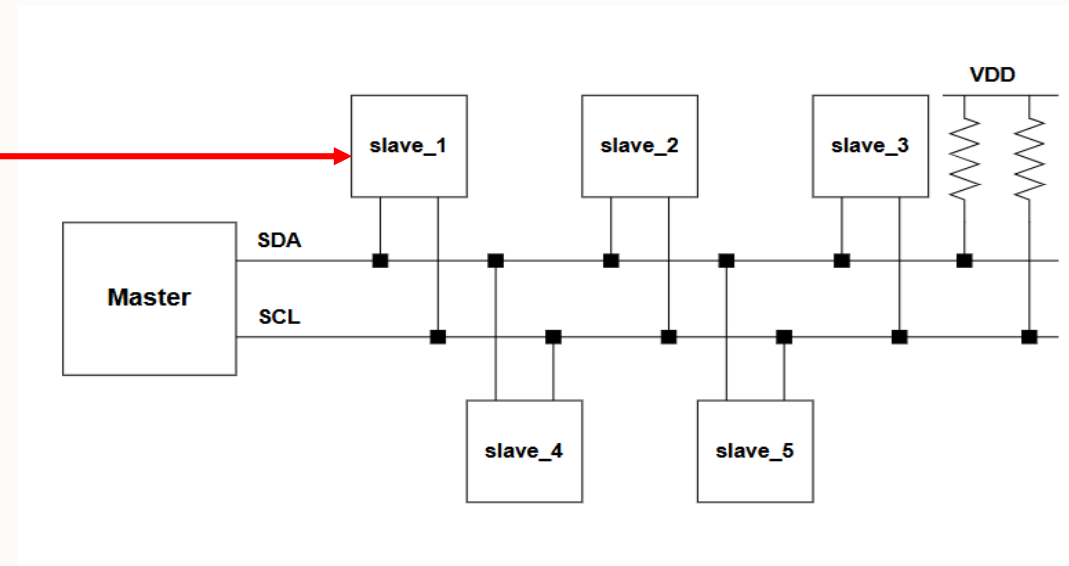
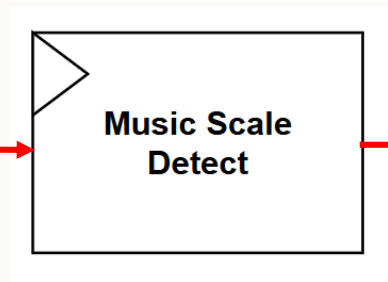
< 실시간 통합 (Integration) >

동기화된 6개 화면을 끊김 없이
하나의 모니터로 출력



Data Integration Flow

Music Scale Detect



01. Camera Image

- 카메라 하단의 '도, 레, 미' 영역 지정
- 지정된 영역 내 빨간색(Red) 객체 검출

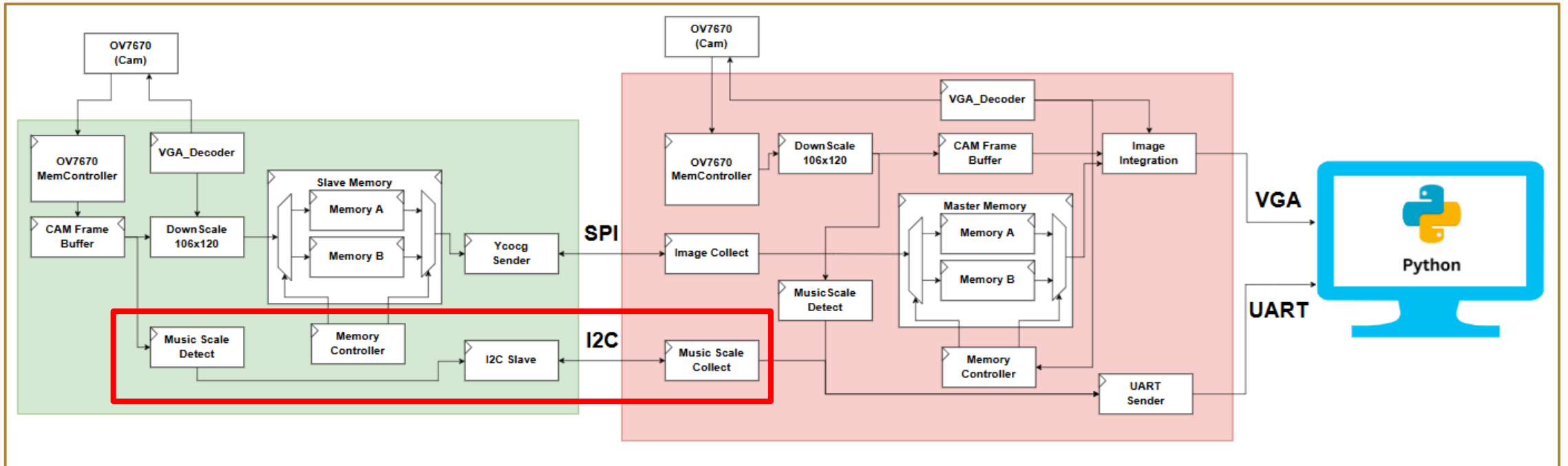
02. Music Scale Detect 모듈

- Music_Scale_Detect: 검출 영역을 2bit 음계 데이터로 출력
- 출력된 2bit 데이터를 I2C slave로 전달

03. I2C Master FSM 모듈

- 마스터 노드의 자체 데이터 (2bit)
- 5개 Slave 음계 데이터 취합 (10bit)
- 총 12bit의 최종 음계 데이터 생성

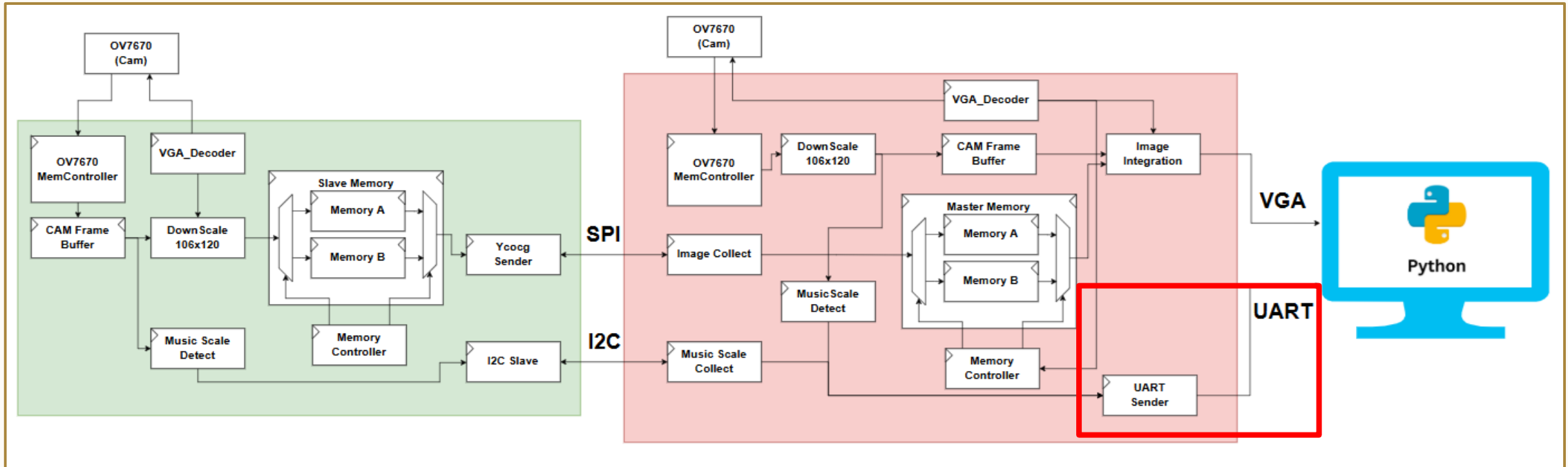
I2C Protocol



Master와 Slave간의 FSM 상태

- IDLE : 2ms 주기 Tick 수신 → 각 Slave 음계(2bit) Read 요청
- SLV1~5 : {6'b0, data}로 master에게 I2C 송신

UART Protocol



3-Byte 패킷 프로토콜 구조

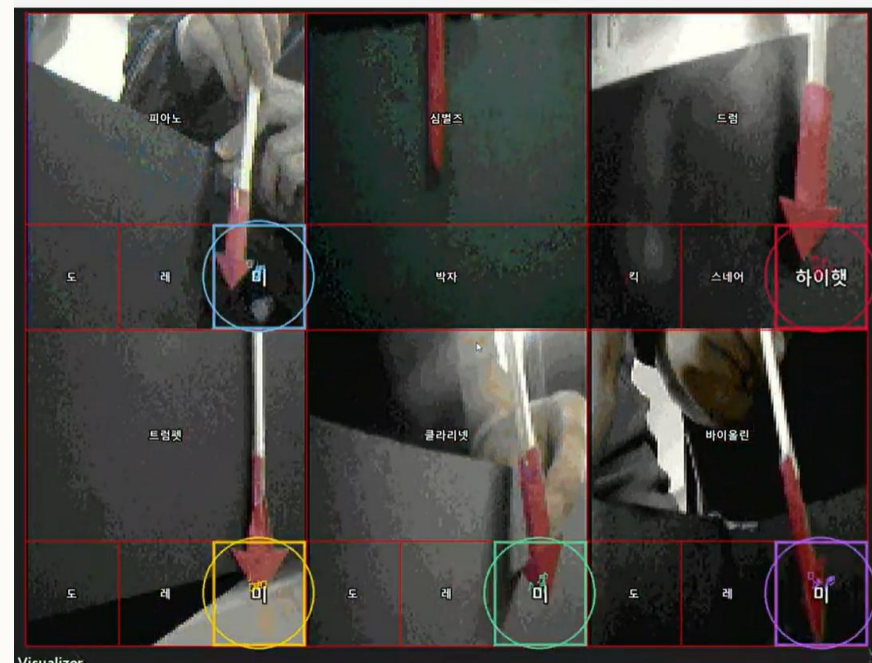
- Start_Header(8'hFF) : 패킷 시작 알림 및 수신 FSM 상태 전환 동기화 토큰
- PHASE1(8-bit) : {2'd0, 드럼, 트럼펫, 피아노} 하위 6비트 병렬 데이터 전송
- PHASE2(8-bit) : {2'd0, 바이올린, 클라리넷, 심벌} 상위 6비트 병렬 데이터 전송



Python / UI

♪ 제 4 악장

Python / UI



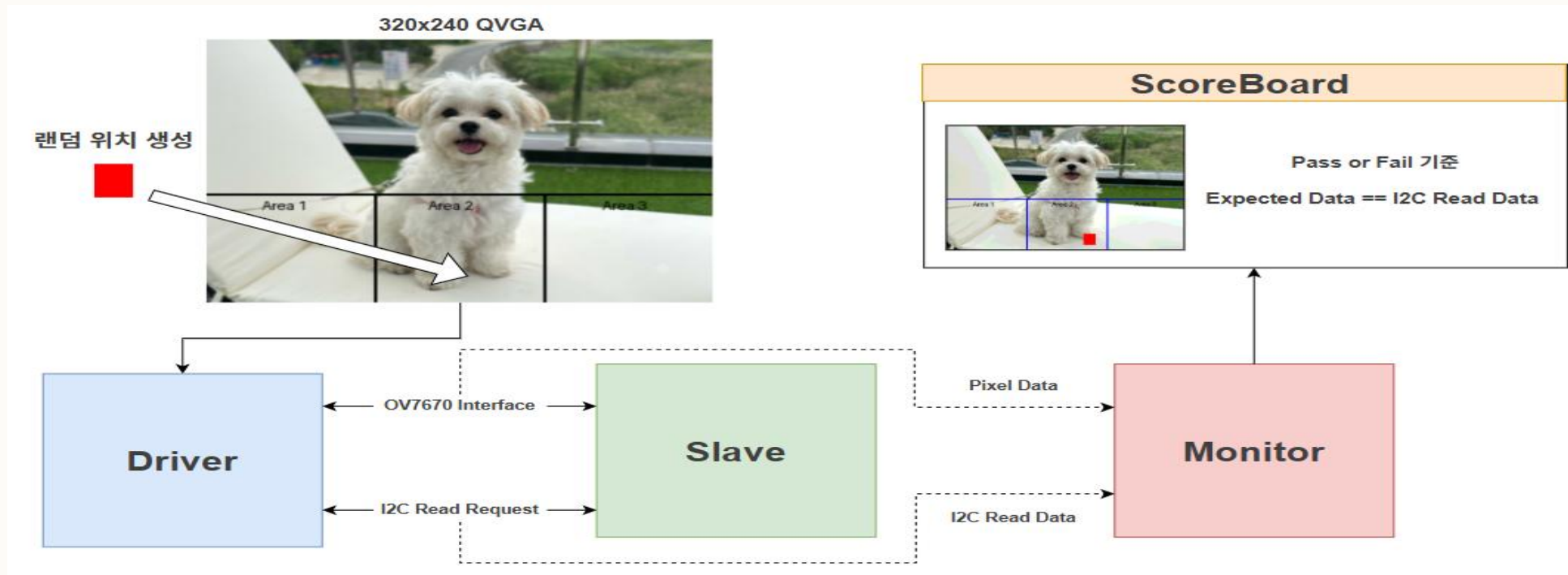
Python 구현 기능

- Interactive UI 제어: START/END 이벤트 기반 화면 전환 및 시스템 상태머신(FSM) 제어
- 실시간 오디오·비주얼 피드백 : 패킷 디코딩 후 소리 출력 및 타깃 위치 동적 이펙트 렌더링
- Vision Overlay 처리: OpenCV 프레임 위 6분할 및 3분할 필터 매핑



UVM Verification

UVM Scenario

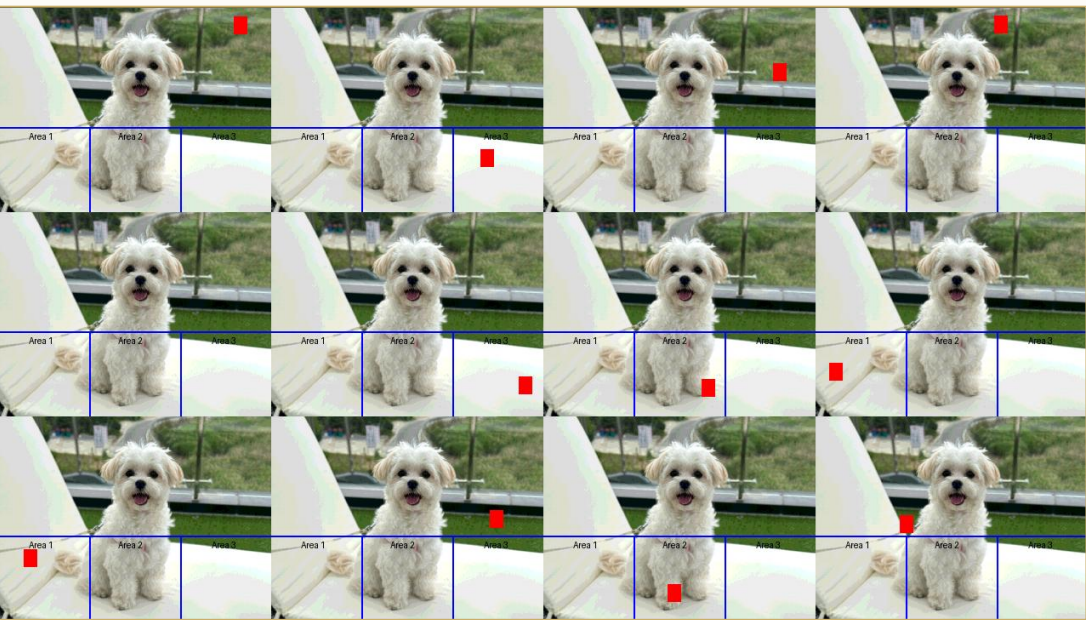


Scenario의 목표

- Red Object를 감지 하는가?
- I2C 통신으로 3가지 Area에 대해 구분하여 데이터를 출력하는가?

UVM Report

500회의 이미지 테스트 수행



- Report

```
=====
| | | VGA Red Detect Summary Report
=====
Result      : ** 검증 완벽 통과 (PASS) **
총 프레임 수: 500
PASS 횟수   : 500
FAIL 횟수   : 0
=====
```

- Coverage

✓	area_1	User	1	1	42
✓	area_2	User	1	1	52
✓	area_3	User	1	1	43
✓	no red	User	1	1	363



Troubleshooting

Troubleshooting - 이미지 송신 안정화



- 파란색과 초록색으로 픽셀이 어긋남
-> YCoCg 데이터 정렬이 밀렸음
- clk_div를 올려도 내려도 깨짐
-> bit단위의 문제가 아니라 byte의 문제

01 Ycocg 데이터가 밀림

[Y3Y2][Y1Y0][CoCg] -> [Y1Y0][CoCg][Y3Y2]로 조립

→ Y : 0 ~ 15 / CoCg : -8 ~ 7 -> CoCg는 +8해서 master로 보낸 후 -8

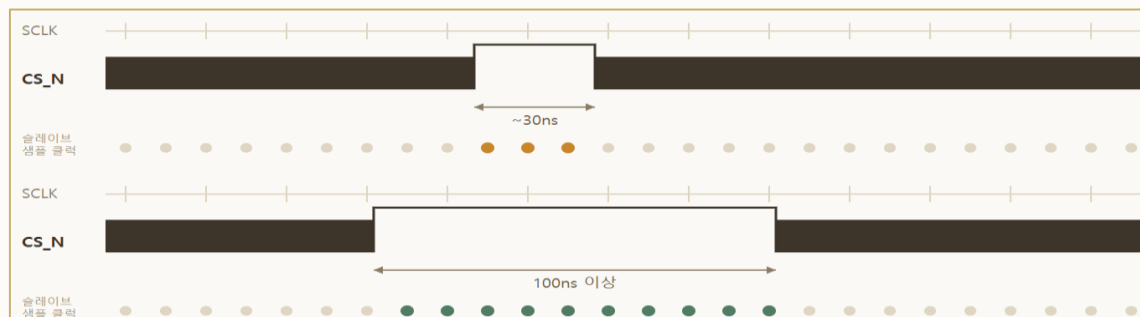
→ Y↓ : B = Y - Cg - Co 포화 → 파란 줄무늬

→ Y↑ : G = Y + Cg 포화 → 초록 줄무늬

02 바이트 간 GAP 추가

master FSM에 GAP 상태를 추가해 cs_n high 구간을 100ns 이상 확보

→ cs 검출 실패로 byte가 밀리는 것을 차단



결과 - 색 왜곡과 수 초간의 정지가 사라지고, 5채널 영상 전송이 안정화.

♪ 제 6 악장

Troubleshooting - 음계 검출 클럭 도메인 통일

```
// ov7670 side
output logic xclk,
input logic pclk,
input logic href,

```

```
VGA_Decoder U_VGA_Decoder (
.clk (clk_100M),
.reset (reset),
.rclk (rclk),
.h_sync (h_sync),

```

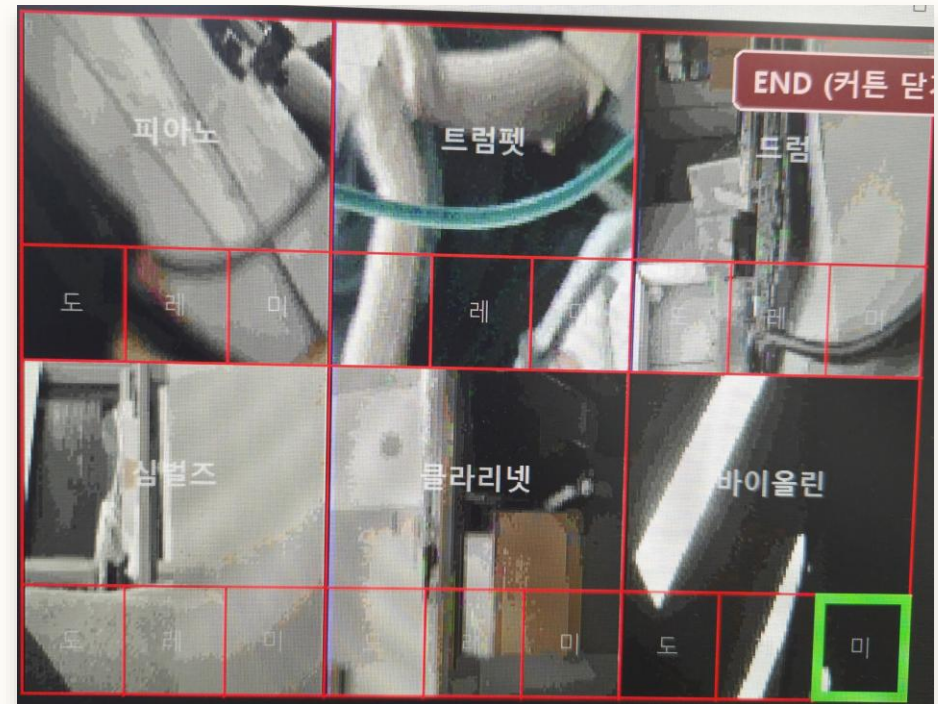
래치 트리거는 VGA v_sync(60Hz), 데이터 누적은 카메라 프레임(30fps).

→ 래치 트리거를 VGA v_sync → 카메라 vsync로 변경

```
(we && red_detect)
```

→ we && red_detect 게이트 - 유효 픽셀인 순간에만 집계하도록 제한

결과 : data_lat이 안정적으로 유지 -> UART가 100% 감지



→ 두 프레임 중 한 번씩 검출이 누락

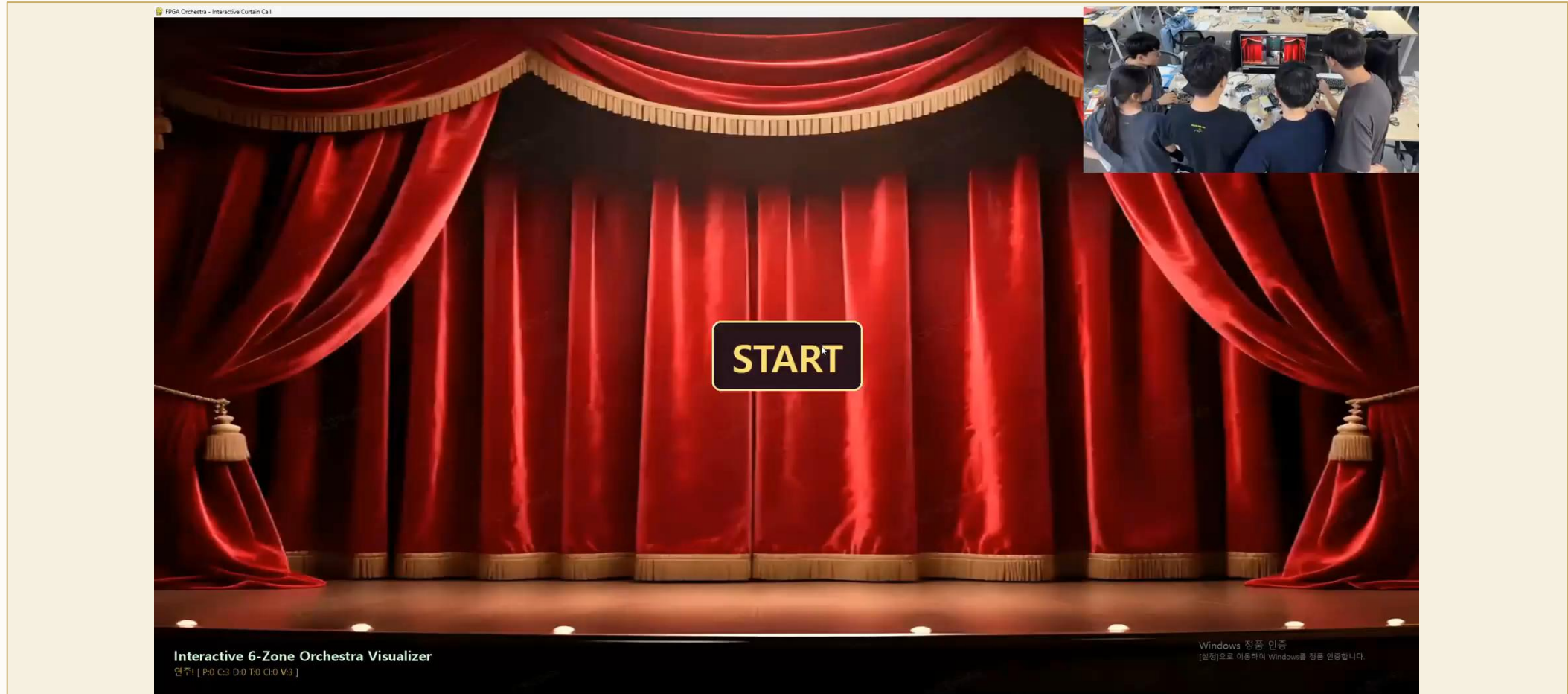
UART가 1초마다 샘플링하는 시점에 따라 결과가 바뀜



Demonstration

♪ 피날레 · FINALE

Video - 땀다땀다 비행기





Reflections

Reflections - 악기 / 역할 분담

01

통신 인터페이스 설계 시, 하드웨어 연결부의 물리적 간섭이 실제 통신에 얼마나 큰 영향을 미치는지 체감하고 배울 수 있었습니다.

VIOLIN 곽은찬 - Python / UART

02

통신 프로토콜 설계 및 디버깅 과정에서, 프로토콜 규격(Spec)에 대한 철저한 숙지와 시뮬레이션을 활용한 단계별 검증이 에러 해결의 핵심임을 깨달았습니다.

CLARINET 안정현 - I2C

03

각자 모듈을 따로 설계하다 보니 통합 단계에서 타이밍 규약이 서로 맞지 않아 예상보다 많은 시간이 들었습니다. 인코더와 디코더처럼 짝을 이루는 모듈일수록 중간중간 설계 방식을 맞춰 나가는 소통이 결과를 좌우한다는 것을 배웠습니다.

DRUM 윤지원 - YCOCG En/decoder

04

FPGA 통합 때 ILA의 장점을 체감하고 배울 수 있었습니다. 이미지 데이터를 마스터가 슬레이브에서 통합하는 과정에서 비정상적 이미지가 나와 난항을 겪었습니다. ILA 활용해 SPI 통신에 문제가 있음을 확인할 수 있는 좋은 계기가 되었습니다.

CYMBALS 이준형 - Master Integration

05

메모리 컨트롤러를 구현하면서 영상 데이터의 읽기, 쓰기 타이밍을 안정적으로 관리하는 것이 실시간 영상처리에서 중요하다는 점을 배웠습니다. 특히 프레임 충돌과 화면 깨짐을 줄이는 과정을 통해 메모리 구조 설계와 신호 동기화에 대한 이해를 높일 수 있었습니다.

TRUMPET 최여지 - Memory Control

06

통신에서 타이밍에 대한 파이프라인의 중요성을 깨달았고, 신호 하나하나의 시간축을 놓치면 결과가 얼마나 쉽게 무너지는지 배웠습니다. 그리고 통합 과정에서의 오류가 발생하였을 때 전체를 보는 것이 아닌 각 영역을 나누어 디버깅을 해보며 원인을 찾는 것의 중요성을 체감하였습니다.

PIANO 최은수 - Slave Integration

— 커튼콜 · CURTAIN CALL —

Thank You

카메라와 FPGA, 그리고 파이썬이 함께 만든 작은 합주
"브레멘 음악대"의 연주를 들어주셔서 감사합니다

악기 음원 소스: 미국 아이오와 대학교 전자음악 스튜디오
오픈소스 악기 음향 데이터베이스 활용

